C++ 核心语法

电子化讲义、分层练习与超详细解析

覆盖范围

变量与常量｜标识符｜整数与浮点数｜sizeof｜ 字符、字符串与布尔类型｜输入输出｜算术、赋值、比较与逻辑运算符｜ if / else if / else

内容来源	C++ 工作草案、Microsoft Learn 与 IBM 编译环境文档
练习构成	10 道单选、5 道多选、5 道填空、4 道综合题
核对基准	C++ 入门：基本类型、表达式、输入输出与选择结构
适用阶段	C++ 入门：基本类型、表达式、输入输出与选择结构
版本日期	2026-07-28

Contents

1 笔记内容电子化与纠错 **1**
1.1 变量：类型、名字、对象和值 . . . 1
1.2 常量：宏、const 与 constexpr . . . 1
1.2.1 宏常量 . . . 1
1.2.2 const 常量 . . . 1
1.2.3 constexpr 常量 . . . 2
1.3 标识符命名规则 . . . 2
1.4 整数类型与可表示范围 . . . 2
1.5 sizeof：测量对象表示所占字节数 . . . 3
1.6 浮点类型与科学计数法 . . . 3
1.6.1 科学计数法 . . . 4
1.7 字符类型、字符编码与转义字符 . . . 4
1.7.1 char 表示一个窄字符代码单元 . . . 4
1.7.2 常见转义序列 . . . 4
1.8 字符串：字符数组与 std::string . . . 4
1.9 布尔类型 bool . . . 5
1.10 使用 std::cin 输入数据 . . . 5
1.11 算术运算符：+ - * / % . . . 6
1.11.1 整数除法 . . . 6
1.11.2 余数运算与恒等式 . . . 6
1.12 前置与后置自增、自减 . . . 6
1.13 赋值与复合赋值 . . . 7
1.14 比较运算符 . . . 7
1.15 逻辑运算符与短路求值 . . . 7
1.15.1 逻辑非 ! . . . 7
1.15.2 逻辑与 && . . . 7
1.15.3 逻辑或 || . . . 8
1.16 if 选择结构 . . . 8
1.16.1 单分支、双分支与多分支 . . . 8
1.16.2 else 与最近的未匹配 if . . . 8
1.16.3 照片中分数程序的正确重写 . . . 9
1.17 照片笔记中的关键纠错总表 . . . 9

2 分层练习题 **10**
2.1 单项选择题（10 题） . . . 10
2.2 多项选择题（5 题） . . . 12
2.3 填空题（5 题） . . . 13
2.4 综合题（4 题） . . . 14

3 答案速查 **16**
3.1 客观题答案 . . . 16
3.2 填空题答案 . . . 16

4 客观题极详细解析 **17**
4.1 单项选择题解析 17
4.2 多项选择题解析 19
4.3 填空题解析 20

5 综合题极详细解析 **21**
5.1 第 21 题：程序逐行追踪 21
5.2 第 22 题：综合查错与重写 22
5.3 第 23 题：带输入校验的成绩等级程序 23
5.4 第 24 题：机器人电池安全监测 24

6 权威资料与核对说明 **27**

使用说明

本讲义不是对照片内容的机械照抄，而是“**忠实电子化 + 标准化纠错**”： 先保留笔记原有的知识顺序，再把会导致错误理解或不可移植代码的表述改正。 例题和练习均围绕照片中的知识边界编写；综合题只少量使用 <iomanip> 等辅助工具，不提前引入循环、数组算法、函数或类。

1 笔记内容电子化与纠错

1.1 变量：类型、名字、对象和值

手写笔记将变量解释为“给一段指定的内存空间起名，方便操作这段内存”。 这对入门阶段很直观，但更精确的说法是： **变量声明把一个名字与某个对象关联；对象具有类型、存储期和值。** 类型决定该对象能表示什么、允许哪些操作，以及通常需要多少存储空间。

基本语法

数据类型　变量名　初始化器;

```cpp
#include <iostream>

int main() {
    int score{0};       // 列表初始化，初值为 0
    double voltage{12.6}; // 浮点变量
    char grade{'A'};    // 字符变量
    bool ready{true};   // 布尔变量

    std::cout << score << ' ' << voltage << ' '
              << grade << ' ' << ready << '\n';
}
```

为什么建议立即初始化

局部的内置类型变量若只写成 int score;，随后在没有先赋值的情况下读取， 会得到不可靠的结果，并可能触发未定义行为。入门阶段优先写 int score{0};，能让“变量当前有什么值”始终清楚。

1.2 常量：宏、const 与 constexpr

1.2.1 宏常量

```cpp
#define DAYS_PER_WEEK 7
```

预处理器会在正式编译前进行宏替换。宏没有普通 C++ 对象那样的类型与作用域， 替换文本也可能产生意外组合。[S8] 因此，除条件编译等预处理用途外， 现代 C++ 通常更偏好有类型的常量。

1.2.2 const 常量

```cpp
const int months_per_year{12};
```

const 表示通过这个对象名不能修改其值。它有明确类型，也服从 C++ 的作用域规则。

1.2.3 constexpr 常量

```
constexpr int days_per_week{7};
```

constexpr 进一步表达“这个值可在编译期作为常量使用”。对本章的纯数值常量，若值在编译时已经确定，优先使用 constexpr。

原笔记代码的两处规范化

- 宏定义行末一般不要写分号：写 #define DAYS 7，而不是 #define DAYS 7;。
- system("pause") 是 Windows 特定做法，不是标准 C++，讲义中的可移植示例将其删除。

1.3 标识符命名规则

标识符是变量名、函数名、类型名等程序实体的名字。对初学者，最安全且可移植的规则是：

1. 只使用英文字母、数字和下划线；
2. 第一个字符不要使用数字；
3. 不能使用 int、if、class 等关键字；
4. 大小写敏感，score 与 Score 是两个名字；
5. 不要自创含双下划线的名字，也不要使用“下划线 + 大写字母”开头的名字，因为这类名字保留给实现使用。[S9][S10]

名字	是否合法	原因
robot_speed	是	由字母与下划线组成，含义清楚
score2	是	数字不在首位
2ndScore	否	首字符是数字
battery-level	否	连字符会被解释为减号
class	否	class 是关键字
__temp	不应使用	含双下划线，属于保留标识符范围

1.4 整数类型与可表示范围

标准有符号整数类型依次为 signed char、short、int、long、long long。标准保证后一个类型提供的存储能力不少于前一个，并规定了最小位宽；**标准没有保证 int 永远是 4 字节，也没有保证 long 永远是 4 字节。**[S1][S16]

若一个有符号整数类型具有 N 个值位，则其范围为

$$\boxed{-2^{N-1} \le x \le 2^{N-1}-1}.$$

推导如下：

- 一共可形成 2^N 个不同的位模式；
- 有符号范围要同时容纳负数、零和正数；
- 负数一侧有 2^{N-1} 个值，正数一侧连同零共有 2^{N-1} 个值；
- 因而最小值是 -2^{N-1}，最大值是 $2^{N-1}-1$。

对应的 N 位无符号整数范围为

$$\boxed{0 \le x \le 2^{N}-1}.$$

类型	标准最小位宽	常见 MSVC x64 大小	在该常见大小下的有符号范围
short	16 位	2 字节	-2^{15} 至 $2^{15}-1$，即 -32768 至 32767
int	16 位	4 字节	-2^{31} 至 $2^{31}-1$
long	32 位	4 字节	-2^{31} 至 $2^{31}-1$
long long	64 位	8 字节	-2^{63} 至 $2^{63}-1$

“常见大小”不能当作语言定理

Windows 的 MSVC 通常采用 long 为 4 字节的数据模型；许多 64 位 Unix-like 环境采用 LP64，此时 long 常为 8 字节。IBM 的 ILP32/LP64 对照文档直接展示了 这种差异。[S11] 写可移植程序时，应使用 sizeof、 std::numeric_limits 或 <cstdint> 中的定宽类型核实需求。

1.5 sizeof：测量对象表示所占字节数

sizeof 的结果是对象或类型所占的**字节数**，结果类型为 std::size_t。对窄字符类型，结果一定为 1；对其他基础类型，大小由实现决定。[S2]

```cpp
#include <iostream>

int main() {
    int value{42};
    std::cout << sizeof(short) << '\n';
    std::cout << sizeof(int) << '\n';
    std::cout << sizeof(value) << '\n';
}
```

标准保证的相对关系可写为

$$\boxed{1 = \mathrm{sizeof(char)} \le \mathrm{sizeof(short)} \le \mathrm{sizeof(int)} \le \mathrm{sizeof(long)} \le \mathrm{sizeof(long\ long)}}.$$

sizeof 的“操作数不求值”

在普通 sizeof(expression) 中，表达式只用于确定类型，通常不会真正执行。 例如 int x{3}; sizeof(x++); 不会让 x 增加。 这属于“未求值操作数”规则，不应把 sizeof 当作会运行括号内代码的函数。

1.6 浮点类型与科学计数法

float、double 和 long double 用来近似表示实数。 在许多常见平台上，float 占 4 字节，double 占 8 字节； 但具体表示格式仍由实现决定。常见 IEEE 754 实现中：

类型	常见大小	常见 digits10	直观含义
float	4 字节	6	至少约 6 位十进制有效数字可不变地表示
double	8 字节	15	至少约 15 位十进制有效数字可不变地表示

这里的“有效数字”不是“小数点后位数”。例如 12345.6 有 6 位有效数字；0.00123456 也有 6 位有效数字。 默认的 std::cout 浮点输出精度通常是 6 位有效数字， 这只是**显示策略**，不能据此断言变量内部只保存 6 位。

1.6.1 科学计数法

$$3e2 = 3 \times 10^{2} = 300,$$
$$3e\text{-}2 = 3 \times 10^{-2} = 0.03.$$

```
float  f1{3e2F};  // 300.0f，后缀 F 表示 float 字面量
double f2{3e-2};  // 0.03，默认浮点字面量类型为 double
```

浮点数是近似值

十进制小数 0.1 往往不能用有限个二进制位精确表示。 因此，浮点计算后直接判断 x == 0.1 可能不符合预期。 本章练习主要考察类型与运算规则，不把浮点误差分析展开到数值算法层面。

1.7 字符类型、字符编码与转义字符

1.7.1 char 表示一个窄字符代码单元

```
char ch{'A'};              // 字符字面量用单引号
int code{static_cast<unsigned char>(ch)};
```

sizeof(char) 必为 1，但 C++ 中的 1 字节不被语言标准强制等于 8 位； 可以通过 CHAR_BIT 查询每字节位数。普通 char 的底层编码也不被强制为 ASCII。多数现代桌面环境对基本拉丁字符使用与 ASCII 兼容的编码，因此 'A' 常见数值是 65，但可移植算法不应依赖这一数值。[S1][S12]

单引号与双引号不能混用

- 'A' 是一个字符字面量；
- "A" 是一个字符串字面量，内部包含 'A' 和结尾的 空字符 '\0'；
- 因而 char c = "A"; 类型不匹配。

1.7.2 常见转义序列

转义序列	含义	示例效果
\n	换行	下一段文字从新行开始
\t	水平制表	移动到下一个制表位置，不等于固定数量空格
\\	反斜杠字符	输出一个 \
\"	双引号字符	可放在字符串字面量内部
\'	单引号字符	可放在字符字面量内部
\0	空字符	C 风格字符串的终止标志

```
std::cout << "Hello,\nworld!\n";
std::cout << "A\tB\n";
std::cout << "C:\\Temp\\file.txt\n";
```

1.8 字符串：字符数组与 std::string

照片中记录了两种字符串表示法。

```cpp
#include <iostream>
#include <string>

int main() {
    char text1[]{"HaHa"};       // C 风格字符数组
    std::string text2{"HaHa"};  // C++ 标准库字符串

    std::cout << text1 << '\n';
    std::cout << text2 << '\n';
}
```

- char text1[]{"HaHa"}; 实际需要 5 个 char： 4 个可见字符加 1 个结尾的 '\0'；
- std::string 是标准库类型，使用时应包含 <string>；
- 对入门字符串操作，std::string 通常更安全、更方便。

1.9 布尔类型 bool

bool 只有两个逻辑值：false 与 true。 默认输出时通常显示为 0 与 1；使用 std::boolalpha 后显示单词。

```cpp
#include <iostream>

int main() {
    bool ready{true};
    bool alarm{false};

    std::cout << ready << ' ' << alarm << '\n';
    std::cout << std::boolalpha << ready << ' ' << alarm << '\n';
}
```

输出为：

```
1 0
true false
```

sizeof(bool) 不由标准固定为 1

许多常见实现中 sizeof(bool) == 1，但标准并不要求它必须如此。 应把“通常为 1 字节”与“语言保证为 1 字节”区分开；后者只适用于窄字符类型。

1.10 使用 std::cin 输入数据

```cpp
#include <iostream>

int main() {
    int value{0};
    std::cout << "Please enter an integer: ";
    std::cin >> value;
    std::cout << "You entered: " << value << '\n';
}
```

>> 是输入提取运算符，<< 是输出插入运算符。 对数字输入，前导空白会被跳过，然后按目标变量的类型解析字符。 如果用户输入的文本不能转换为目标类型，输入流会进入失败状态； 综合题 3 会演示最基本的失败检查。

```
bool flag{};
std::cin >> std::boolalpha >> flag; // 可输入 true 或 false
```

1.11 算术运算符：+ - * / %

运算符	名称	核心规则
+	加法	两个算术操作数相加
-	减法	第一个操作数减去第二个操作数
*	乘法	两个算术操作数相乘
/	除法	整数除法丢弃小数部分；浮点除法保留浮点结果
%	余数	只适用于整数或无作用域枚举；不能用于 double

1.11.1 整数除法

$$7/2 = 3,$$
$$-7/2 = -3.$$

整数商向零截断，也就是把代数商的小数部分直接丢弃。[S3] 因此 double x = 7 / 2; 先进行整数除法得到 3，再转换为 double，最终是 3.0，不是 3.5。 若要得到 3.5，至少一个操作数必须是浮点数： 7.0 / 2 或 static_cast<double>(7) / 2。

1.11.2 余数运算与恒等式

在商可表示且除数不为 0 时，整数除法满足

$$\boxed{a = (a/b) \times b + a \bmod b}.$$

例如 $17/5 = 3$，$17 \bmod 5 = 2$，所以

$$17 = 3 \times 5 + 2.$$

除数为 0

对整数 a / 0 或 a % 0，程序行为未定义。 这不是“结果等于 0”或“结果等于无穷大”，而是语言不再规定程序应怎样运行。[S3]

1.12 前置与后置自增、自减

写法	表达式产生的值	对象最终变化
++x	先加 1，再产生新值	x 增加 1
x++	先产生旧值，再完成加 1	x 增加 1
--x	先减 1，再产生新值	x 减少 1
x--	先产生旧值，再完成减 1	x 减少 1

```
int x{3};
int old{x++}; // old = 3，随后 x = 4
int now{++x}; // 先令 x = 5，再令 now = 5
```

单独写成一条语句时，++x; 与 x++; 都会让 x 增加 1； 差异只有在表达式的值被继续使用时才显现。标准规定前置 ++x 在内置类型上等价于 x += 1，后置形式则产生原值的副本。[S4][S5]

1.13 赋值与复合赋值

运算符	示例	效果
=	a = 2	把右侧值赋给左侧可修改对象
+=	a += 2	近似理解为 a = a + 2
-=	a -= 2	近似理解为 a = a - 2
*=	a *= 2	近似理解为 a = a * 2
/=	a /= 2	近似理解为 a = a / 2
%=	a %= 2	近似理解为 a = a % 2

更严格地说，E1 op= E2 与 E1 = E1 op E2 的值效果相同， 但复合赋值只求值一次左操作数。[S6] 同时要记住：赋值号 = 不是数学中的等号； 判断相等要用 ==。

1.14 比较运算符

运算符	含义	示例与结果
==	相等	4 == 3 为 false
!=	不相等	4 != 3 为 true
<	小于	4 < 3 为 false
>	大于	4 > 3 为 true
>=	大于等于	4 >= 3 为 true
<=	小于等于	4 <= 3 为 false

比较结果的类型是 bool。[S7]

C++ 不能照抄数学链式不等式

表达式 650 <= score < 700 会按从左到右理解为

$$(650 <= \text{score}) < 700.$$

括号内先产生 false 或 true，再转换为 0 或 1； 无论 0 还是 1 都小于 700，因此这个表达式几乎总为真。 正确写法是：

$$\boxed{\text{score} >= 650 \;\&\&\; \text{score} < 700}.$$

标准草案也明确用 a < b < c 说明其含义不是数学区间判断。[S7]

1.15 逻辑运算符与短路求值

1.15.1 逻辑非 !

$$!\text{true} = \text{false}, \qquad !\text{false} = \text{true}.$$

1.15.2 逻辑与 &&

A	B	A && B
false	false	false
false	true	false
true	false	false
true	true	true

只有两边都为真，结果才为真。若左操作数已经为假，右操作数不会求值。

1.15.3 逻辑或 ||

A	B	$A \,\|\|\, B$
false	false	false
false	true	true
true	false	true
true	true	true

只要至少一边为真，结果就为真。若左操作数已经为真，右操作数不会求值。[S13][S14]

```
int denominator{0};
if (denominator != 0 && 10 / denominator > 2) {
  std::cout << "condition satisfied\n";
}
```

这里左侧为假，右侧 10 / denominator 不会执行，因此避免了除以 0。 这就是短路求值最常见的安全用途之一。

1.16 if 选择结构

1.16.1 单分支、双分支与多分支

```
if (condition) {
  // condition 为 true 时执行
}

if (condition) {
  // true 分支
} else {
  // false 分支
}

if (condition1) {
  // 分支 1
} else if (condition2) {
  // 分支 2
} else {
  // 其余情况
}
```

条件会被按布尔语境转换：真时执行第一个子语句，假且存在 else 时执行第二个子语句。[S15] 虽然只有一条语句时可省略花括号，但入门阶段建议始终写花括号， 以降低缩进与实际控制关系不一致的风险。

1.16.2 else 与最近的未匹配 if

```
if (x > 0)
  if (x > 10)
    std::cout << "A";
  else
    std::cout << "B";
```

这里的 else 与内层 if (x > 10) 匹配，而不是与外层 if (x > 0) 匹配。标准规则是：else 归属最近的、尚未拥有 else 的 if。[S15]

1.16.3 照片中分数程序的正确重写

```cpp
#include <iostream>

int main() {
    int score{0};
    std::cout << "Please enter a score: ";
    std::cin >> score;

    if (score >= 700) {
        std::cout << "Level A\n";
    } else if (score >= 650) {
        std::cout << "Level B\n";
    } else if (score >= 600) {
        std::cout << "Level C\n";
    } else {
        std::cout << "Keep working\n";
    }
}
```

为什么后两个条件不必再写上界

程序只会进入第一个为真的分支。执行到 else if (score >= 650) 时， 前面的 score >= 700 已经为假，所以此时自动知道 $score < 700$。因此该分支实际覆盖

$$650 \le score < 700.$$

同理，第三个分支实际覆盖 $600 \le score < 650$。 把阈值从高到低排列，既简洁又不重叠。

1.17 照片笔记中的关键纠错总表

照片中的简化或问题	准确结论	原因
int 固定 4 字节	标准只规定最小位宽与类型大小顺序	具体大小由实现与数据模型决定
long 固定 4 字节	Windows/ MSVC 常为 4 字节，LP64 环境常为 8 字节	同一份 C++ 源码可面向不同 ABI
bool 占 1 字节	常见但不是标准保证	只有窄字符类型的 sizeof 被保证为 1
char 就是 ASCII	普通字符编码由实现决定	现代环境常与 ASCII 兼容，但程序不应无条件假设
float “精确 7 位”	常见实现约有 6 位 digits10，显示与存储精度也要区分	“有效数字”“往返精度”“默认显示位数”是不同概念
650 <= score < 700	应写 score >= 650 && score < 700	关系运算从左到右结合，第一次比较先变成布尔值
% 可对小数使用	% 只接受整数或无作用域枚举	浮点余数需使用标准库函数，超出本章范围
system("pause") 是必需语句	它不是标准 C++，可移植示例不需要它	这是特定系统命令，不属于语言语法

2 分层练习题

命题说明

本套题共 24 题：单项选择 10 题、多项选择 5 题、填空 5 题、综合题 4 题。题目均为依据权威资料与入门阶段高频错误重新编写的原创练习，不是对网页题目的逐字复制。难度标签只表示本章内部的相对梯度。

2.1 单项选择题（10 题）

1. 标识符

下列名字中，可作为安全、常规的 C++ 标识符的是（　）。　难度：基础

A. 2ndScore

B. class

C. robot_speed

D. battery-level

2. sizeof 的标准保证

下列结论中，C++ 语言标准保证成立的是（　）。　难度：基础

A. sizeof(int) == 4

B. sizeof(bool) == 1

C. sizeof(char) == 1

D. sizeof(long) == 4

3. 整数除法与余数

下列程序的输出是（　）。　难度：基础

```
int a{7};
int b{2};
std::cout << a / b << ' ' << a % b;
```

A. 3 1

B. 3.5 1

C. 3 0

D. 4 1

4. 运算发生的类型

下列程序的输出是（　）。　难度：基础

```
double x{7 / 2};
double y{7.0 / 2};
std::cout << x << ' ' << y;
```

A. 3.5 3.5

B. 3 3

C. 3 3.5

D. 程序无法编译

5. 前置与后置自增

下列程序的输出是（ ）。 难度：中等

```
int x{3};
int y{x++};
int z{++x};
std::cout << x << ' ' << y << ' ' << z;
```

A. 3 3 3

B. 4 3 4

C. 5 3 5

D. 5 4 5

6. 逻辑表达式

下列程序的输出是（ ）。 难度：中等

```
int score{640};
bool in_range{score >= 600 && score < 650};
std::cout << std::boolalpha << in_range;
```

A. 0

B. 1

C. false

D. true

7. 区间判断

要准确表示数学区间 $650 \leq score < 700$，正确的 C++ 条件是（ ）。 难度：中等

A. 650 <= score < 700

B. score >= 650 && score < 700

C. score >= 650 || score < 700

D. score > 650 && score <= 700

8. 宏与有类型常量

关于 #define DAYS 7 与 constexpr int days{7};， 下列说法正确的是（ ）。 难度：中等

A. 两者都会创建一个完全相同的 int 对象

B. 宏进行预处理替换；constexpr 对象有类型并遵守作用域

C. 宏只能在 main 函数内定义

D. constexpr 变量可以在运行时被重新赋值

9. 复合赋值

下列程序的输出是（ ）。 难度：提高

```
int a{5};
a *= 2 + 3;
std::cout << a;
```

A. 7

B. 13

C. 25

D. 30

10. else 的归属

设 x == 5，下列代码输出（ ）。 难度：提高

```
if (x > 0)
  if (x > 10)
    std::cout << "A";
  else
    std::cout << "B";
```

A. A

B. B

C. 没有输出

D. 语法错误

2.2 多项选择题（5 题）

11. 基础类型的可移植结论

下列结论中，标准保证成立的有（ ）。 难度：中等

A. sizeof(char) == 1

B. sizeof(short) <= sizeof(int)

C. 所有平台的 int 都是 4 字节

D. 若有符号整数宽度为 N，其范围是 -2^{N-1} 至 $2^{N-1}-1$

12. 字符与字符串声明

在已经包含 <string> 的前提下，下列声明正确的有（ ）。 难度：中等

A. char c{'A'};

B. char c{"A"};

C. std::string s{"A"};

D. char s[]{"A"};

13. 除法与余数的错误风险

设 int a{7}; int b{0};，下列说法正确的有（ ）。 难度：中等

A. 求值 a / b 会触发未定义行为

B. 求值 a % b 会触发未定义行为

C. 表达式 3.0 % 2 不符合内置余数运算符的类型要求

D. 表达式 a / 2 的值是 3

14. 短路求值

设 int x{0};，分别独立判断下列说法，正确的有（ ）。 难度：提高

A. x != 0 && 10 / x > 1 安全地产生 false

B. x == 0 || ++x > 0 产生 true，且 x 仍为 0

C. !x 产生 true

D. x++ && ++x 求值后 x == 2

15. if 结构

下列说法正确的有（ ）。 难度：提高

A. 没有花括号时，else 与最近的未匹配 if 结合

B. if 的条件表达式必须原本就是 bool 类型，整数不允许

C. 即使分支只有一条语句，使用花括号也能降低维护错误

D. 650 <= score < 700 与 score >= 650 && score < 700 完全等价

2.3 填空题（5 题）

16. 有类型常量

请补全一个值为 12、初始化后不能修改的整型常量：

_____ months_per_year{12};

难度：基础

17. 类型大小

运算符 _____用于获得类型或对象占用的字节数； 其结果类型是 _____。 难度：基础

18. 复合赋值追踪

填写程序的最终输出：_____。 难度：中等

```
int a{11};
a /= 4;
a += 2;
std::cout << a;
```

19. 闭区间条件

要判断温度 temperature 是否位于闭区间 $[-10, 50]$，条件应写为：

_____.

难度：中等

20. 自增表达式追踪

填写程序的输出：_____。 难度：提高

```
int n{2};
std::cout << n++ << ' ' << ++n;
```

2.4 综合题（4 题）

21. 程序逐行追踪

不运行程序，写出完整输出，并逐行说明每个变量的变化。 难度：中等

```
#include <iostream>

int main() {
    int a{17};
    int b{5};
    int q{a / b};
    int r{a % b};
    int old{b++};
    int now{++b};
    bool ok{(q == 3 && r == 2) || (old > now)};
    a += b * 2;

    std::cout << a << ' ' << b << ' '
              << q << ' ' << r << ' '
              << old << ' ' << now << ' '
              << std::boolalpha << ok << '\n';
}
```

22. 综合查错与重写

下列程序试图读取分数并输出等级，但含有多处语法、类型或逻辑问题。 请至少找出 8 处问题，说明原因，并写出一份能正确编译、逻辑清晰的版本。 难度：提高

```
#include <iostream>
using namespace std;

#define DAYS 7;

int main() {
    const int months;
    char grade = "A";
    string name = 'Robot';
    int score;

    cout << "score: "
    cin << score;

    if (650 <= score < 700)
        cout << "A";
    else if (score = 600)
        cout << "B";

    cout << 10 % 2.5;
}
```

23. 带输入校验的成绩等级程序

编写一个程序： 难度：提高

1. 读取一个整数分数；
2. 若输入无法转换为整数，输出 Invalid input 并结束；
3. 若分数不在 $[0, 100]$，输出 Out of range；
4. 否则按下列规则输出等级：

区间	等级
$90 \le s \le 100$	A
$80 \le s < 90$	B
$70 \le s < 80$	C
$60 \le s < 70$	D
$0 \le s < 60$	F

要求使用从高到低的 if -- else if -- else 链，并给出边界测试。

24. 机器人电池安全监测

某移动机器人需要根据电池端电压 U、工作电流 I、电池温度 T 和预计任务时长 t 判断状态。 难度：综合

1. 输入四个 double：U（V）、I（A）、T（°C）、t（min）；
2. 若 $U \le 0$、$I < 0$ 或 $t \le 0$，输出 Invalid data；
3. 否则计算功率与预计能量：

$$P = UI, \qquad E = P \cdot \frac{t}{60};$$

4. 状态判定如下：

$$\begin{aligned} &\text{Critical: } U < 10.5 \quad \text{或} \quad T > 60 \quad \text{或} \quad I > 20; \\ &\text{Warning: } U < 11.1 \quad \text{或} \quad T > 45 \quad \text{或} \quad I > 15; \\ &\text{Normal: 其余情况。} \end{aligned}$$

请写出完整程序，解释公式单位、分支顺序及逻辑运算，并至少给出 4 组测试。

3 答案速查

3.1 客观题答案

题号	答案	题号	答案
1	C	**2**	C
3	A	**4**	C
5	C	**6**	D
7	B	**8**	B
9	C	**10**	B
11	A、B、D	**12**	A、C、D
13	A、B、C、D	**14**	A、B、C
15	A、C		

3.2 填空题答案

题号	答案
16	const int（若强调编译期常量，constexpr int 也成立）
17	sizeof；std::size_t
18	4
19	temperature >= -10 && temperature <= 50
20	2 4

4 客观题极详细解析

4.1 单项选择题解析

第 1 题：答案 C

核心判断

robot_speed 只由字母和下划线组成，首字符不是数字，也不是关键字。

- A 错：2ndScore 以数字开头；
- B 错：class 是 C++ 关键字；
- C 对：满足安全、常规的命名规则；
- D 错：- 是减法运算符，不属于这种标识符的组成字符。

第 2 题：答案 C

核心判断

C++ 保证对窄字符类型应用 sizeof 的结果为 1，因此 sizeof(char) == 1。

- A 错：int 常为 4 字节，但标准允许其他合规实现；
- B 错：bool 常为 1 字节，但不属于必须为 1 的保证；
- C 对：这是标准的明确保证；
- D 错：long 在 MSVC x64 常为 4 字节，在 LP64 环境常为 8 字节。

第 3 题：答案 A

$$7/2 = 3, \qquad 7 \bmod 2 = 1.$$

两个操作数都是 int，所以 / 执行整数除法，小数部分被丢弃。 又因为

$$7 = 3 \times 2 + 1,$$

余数为 1。因此输出 3 1。

- B 错：7 / 2 不会产生 3.5；
- C 错：7 不是 2 的整数倍，余数不为 0；
- D 错：整数除法向零截断，不是四舍五入。

第 4 题：答案 C

初始化 x 时，先计算右侧 7 / 2。计算发生时两个操作数都是 int，结果为整数 3；之后才转换成 double 存入 x。 所以 $x = 3.0$。

初始化 y 时，7.0 是 double，另一个操作数 2 会参与通常算术转换， 除法按浮点数完成：

$$7.0/2 = 3.5.$$

默认输出通常显示 3 3.5，故选 C。

第 5 题：答案 C

执行语句	x 执行前	产生的表达式值	x 执行后
int y{x++}; int z{++x};	3 4	3，赋给 y 5，赋给 z	4 5

最终 $x=5$、$y=3$、$z=5$，输出 5 3 5。 选项 D 把后置自增误认为“先加再取值”。

第 6 题：答案 D

依次计算：

$$640 \geq 600 \Rightarrow \text{true}, \qquad 640 < 650 \Rightarrow \text{true}.$$

逻辑与要求两边都为真，所以

$$\text{true \&\& true} = \text{true}.$$

std::boolalpha 把布尔值显示为单词，因此输出 true，而不是数字 1。

第 7 题：答案 B

数学区间 $650 \leq score < 700$ 同时要求：

$$\begin{cases} score \geq 650, \\ score < 700. \end{cases}$$

“同时满足”对应逻辑与 &&，所以正确写法为

$$\boxed{\text{score >= 650 \&\& score < 700}}.$$

- A 错：C++ 将其理解为 (650 <= score) < 700；
- C 错：逻辑或会让几乎所有数满足至少一边，区间被错误扩大；
- D 错：它表示 $650 < score \leq 700$，边界方向与题意不同。

第 8 题：答案 B

#define DAYS 7 建立预处理宏，使用处会发生记号替换； constexpr int days{7}; 声明一个有明确 int 类型、遵守作用域的常量对象。

- A 错：宏不是普通 int 对象；
- B 对：准确指出处理阶段、类型和作用域的差异；
- C 错：宏可在函数外定义，而且通常正是在文件层面定义；
- D 错：constexpr 对象不能被普通赋值修改。

第 9 题：答案 C

a *= 2 + 3; 等价地理解为

$$a = a \times (2+3).$$

先算括号意义上的右侧加法：

$$2+3=5,$$

再计算

$$a = 5 \times 5 = 25.$$

因此输出 25。它不是 a = a * 2 + 3；复合赋值的右操作数是完整的 2 + 3。

第 10 题：答案 B

当 $x = 5$ 时：

1. 外层条件 x > 0 为真，进入内层；
2. 内层条件 x > 10 为假；
3. else 与最近的内层 if 匹配，于是输出 B。

   把花括号补全后，实际结构是：

```
if (x > 0) {
  if (x > 10) {
    std::cout << "A";
  } else {
    std::cout << "B";
  }
}
```

4.2 多项选择题解析

第 11 题：答案 A、B、D

- A 对：sizeof(char) == 1 是标准保证；
- B 对：标准保证 sizeof(short) $\leq$ sizeof(int)；
- C 错：4 字节只是广泛实现，不是所有合规实现必须采用；
- D 对：有符号 N 位整数范围是 $[-2^{N-1},\ 2^{N-1}-1]$。

  以 $N = 16$ 为例：

$$-2^{15} = -32768, \qquad 2^{15} - 1 = 32767.$$

之所以正数上界少 1，是因为非负一侧还要包含 0。

第 12 题：答案 A、C、D

- A 对：'A' 是字符字面量，可初始化 char；
- B 错："A" 是字符串字面量，不能直接初始化单个 char；
- C 对：std::string 可由字符串字面量构造；
- D 对：数组大小可由初始化字符串推导，实际长度为 2： 'A' 与 '\0'。

第 13 题：答案 A、B、C、D

- A 对：整数除数 b 为 0，a / b 是未定义行为；
- B 对：a % b 同样要求除数不为 0；
- C 对：内置 % 要求整数或无作用域枚举操作数， 3.0 是 double；
- D 对：7 / 2 为整数除法，向零截断得到 3。

为什么本题四项都可选

多选题不要求一定存在错误选项。本题刻意把“除以零”“浮点取余”“正常整数除法” 放在一起，检查能否分别判断运行时未定义行为、编译期类型错误与合法表达式。

第 14 题：答案 A、B、C

四个表达式彼此独立，每次都从 $x = 0$ 开始。

- A 对：x != 0 为假，&& 短路，右侧除法不执行；
- B 对：x == 0 为真，|| 短路，++x 不执行， 所以结果为真且 $x = 0$；
- C 对：整数 0 在布尔语境中为假，逻辑非后为真；
- D 错：x++ 先产生旧值 0，因此逻辑与左侧为假； 后置自增仍让 x 变为 1，但右侧 ++x 被短路。 最终 $x = 1$，不是 2。

第 15 题：答案 A、C

- A 对：这是 else 的正式结合规则；
- B 错：条件可进行“按布尔语境转换”，例如 0 表示假，非 0 整数表示真；
- C 对：花括号使控制范围显式可见，后续添加语句时更安全；
- D 错：链式关系表达式按左结合计算，不等价于逻辑与。

4.3 填空题解析

第 16 题

标准答案：

```
const int months_per_year{12};
```

const 修饰 int 对象，表示初始化后不能通过该名字修改。 如果希望明确表达“编译期常量”，也可写：

```
constexpr int months_per_year{12};
```

第 17 题

答案为 sizeof 与 std::size_t。 sizeof(T) 返回一个足以表示对象大小的无符号整数类型； 标准库把这个类型命名为 std::size_t。

第 18 题

初始 $a = 11$。

$$a \mathrel{/}= 4 \quad \Longrightarrow \quad a = 11/4 = 2.$$

因为 a 是 int，所以 $11/4$ 向零截断为 2。随后：

$$a \mathrel{+}= 2 \quad \Longrightarrow \quad a = 2 + 2 = 4.$$

最终输出 4。

第 19 题

闭区间 $[-10, 50]$ 包含两个端点，因此应写：

```
temperature >= -10 && temperature <= 50
```

逻辑与不可换成逻辑或。若使用 ||，例如温度 100 仍满足 100 >= -10，从而被误判为区间内。

第 20 题

初始 $n = 2$。

1. n++ 产生旧值 2 用于第一次输出，然后 n 变为 3；
2. ++n 先把 n 增加到 4，再产生 4 用于第二次输出。

   所以输出：

```
2 4
```

5 综合题极详细解析

5.1 第 21 题：程序逐行追踪

最终输出

```
31 7 3 2 5 7 true
```

第一步：初始化

$$a = 17, \qquad b = 5.$$

第二步：整数商与余数

$$q = 17/5 = 3, \qquad r = 17 \bmod 5 = 2.$$

验证除法恒等式：

$$(17/5) \times 5 + (17 \bmod 5) = 3 \times 5 + 2 = 17.$$

第三步：两种自增

语句	b 执行前	赋给新变量的值	b 执行后
int old{b++}; int now{++b};	5 6	$old = 5$ $now = 7$	6 7

后置自增先给出旧值，前置自增先修改再给出新值，所以最终

$$old = 5, \qquad now = 7, \qquad b = 7.$$

第四步：逻辑表达式

$$\text{ok} = (\text{q} == 3 \;\&\&\; \text{r} == 2) \;||\; (\text{old} > \text{now}).$$

逐项代入：

$$q == 3 \Rightarrow 3 == 3 \Rightarrow \text{true},$$
$$r == 2 \Rightarrow 2 == 2 \Rightarrow \text{true},$$
$$\text{true} \;\&\&\; \text{true} \Rightarrow \text{true}.$$

逻辑或左侧已经为真，按照短路规则，右侧 old > now 不必再求值；整个表达式为 true。 即使手工核对右侧，$5 > 7$ 也是假，不影响 true || false = true。

第五步：复合赋值

$$\begin{aligned} a \mathrel{+}= b \times 2, \\ &= 17 + 7 \times 2, \\ &= 17 + 14, \\ &= 31. \end{aligned}$$

最终变量状态为：

变量	a	b	q	r	old	now	ok
值	31	7	3	2	5	7	true

5.2 第 22 题：综合查错与重写

问题清单

序号	原代码	问题与修正
1	#define DAYS 7;	宏替换列表包含分号，易在使用处产生多余分号；应去掉，或改用 constexpr int days_per_week{7};
2	const int months;	const 对象必须初始化，例如 const int months{12};
3	char grade = "A";	"A" 是字符串字面量；单个字符应写 'A'
4	使用 string	缺少 #include <string>，且最好写完整名字 std::string
5	'Robot'	单引号用于字符字面量，字符串应写 "Robot"
6	int score;	虽可声明，但立即初始化成 int score{0}; 更安全、更清楚
7	cout << "score: "	语句末缺少分号
8	cin << score;	输入方向写反；应为 std::cin >> score;
9	650 <= score < 700	C++ 链式关系运算不表示数学区间；应写 score >= 650 && score < 700
10	score = 600	= 是赋值，不是相等比较；相等比较是 ==。 更合理的是按阈值从高到低组织分支
11	10 % 2.5	% 不能接受 double 操作数；若只演示整数余数，可写 10 % 2
12	无输入失败检查	如果用户输入非整数，后续分支没有可靠意义；可先检查 if (!(std::cin >> score))

一份正确、清晰的重写

```cpp
#include <iostream>
#include <string>

constexpr int days_per_week{7};

int main() {
    const int months_per_year{12};
    char grade{'D'};
    std::string name{"Robot"};
    int score{0};

    std::cout << "score: ";
    if (!(std::cin >> score)) {
        std::cout << "Invalid input\n";
        return 1;
    }
```

```

    if (score >= 700) {
        grade = 'A';
    } else if (score >= 650) {
        grade = 'B';
    } else if (score >= 600) {
        grade = 'C';
    } else {
        grade = 'D';
    }

    std::cout << name << " grade: " << grade << '\n';
    std::cout << "days per week: " << days_per_week << '\n';
    std::cout << "months per year: " << months_per_year << '\n';
    std::cout << "10 % 2 = " << 10 % 2 << '\n';
}
```

分支为什么按阈值从高到低

若先判断 score >= 600，那么 700、800 等更高分数也立即进入该分支，后续更高等级永远没有机会判断。按 700、650、600 的顺序排列，每个 else if 都自动获得前面条件为假的信息，从而形成互不重叠的区间。

5.3 第 23 题：带输入校验的成绩等级程序

参考程序

```
#include <iostream>

int main() {
    int score{0};
    std::cout << "Please enter a score (0-100): ";

    if (!(std::cin >> score)) {
        std::cout << "Invalid input\n";
        return 1;
    }

    if (score < 0 || score > 100) {
        std::cout << "Out of range\n";
        return 1;
    }

    char grade{'F'};
    if (score >= 90) {
        grade = 'A';
    } else if (score >= 80) {
        grade = 'B';
    } else if (score >= 70) {
        grade = 'C';
    } else if (score >= 60) {
        grade = 'D';
    } else {
        grade = 'F';
    }

    std::cout << "Grade: " << grade << '\n';
```

```
}
```

输入校验为什么分成两层

1. **格式校验：** std::cin >> score 尝试把输入转换成整数。 若用户输入 abc，提取失败，流对象在条件中表现为假； 外层逻辑非 ! 把它变为真，于是输出 Invalid input。
2. **范围校验：** 输入 -3 或 120 都能成功转换为整数， 但不属于合法分数。条件

$$score < 0 \quad \text{或} \quad score > 100$$

对应 score < 0 || score > 100。

等级区间是怎样自然形成的

假设程序执行到 else if (score >= 80)： 前一个 score >= 90 已经为假，所以已知 $score < 90$； 当前条件又要求 $score \geq 80$。两条信息合并即

$$80 \leq score < 90.$$

这正是 B 等级区间，因此不必重复写上界。

边界测试

输入	预期输出	覆盖目的
-1	Out of range	下界外
0	Grade: F	合法最小值
59	Grade: F	F 上边界内
60	Grade: D	D 下边界
69	Grade: D	D 上边界内
70	Grade: C	C 下边界
79	Grade: C	C 上边界内
80	Grade: B	B 下边界
89	Grade: B	B 上边界内
90	Grade: A	A 下边界
100	Grade: A	合法最大值
101	Out of range	上界外
abc	Invalid input	非整数输入

5.4 第 24 题：机器人电池安全监测

公式与单位

电功率公式为

$$\boxed{P = UI}.$$

单位关系是

$$\mathrm{V} \times \mathrm{A} = \mathrm{W}.$$

所以当电压 U 以伏特输入、电流 I 以安培输入时，功率 P 的单位是瓦特。

能量等于功率乘时间：

$$E = P\Delta t.$$

题目给出的时间单位是分钟，而瓦时要求时间单位为小时。因为

$$1\ \mathrm{h} = 60\ \mathrm{min},$$

所以

$$\Delta t_{\mathrm{h}} = \frac{t_{\mathrm{min}}}{60},$$

进而得到

$$\boxed{E_{\mathrm{Wh}} = P_{\mathrm{W}} \times \frac{t_{\mathrm{min}}}{60}}.$$

参考程序

```cpp
#include <iostream>

int main() {
    double voltage{0.0};
    double current{0.0};
    double temperature{0.0};
    double minutes{0.0};

    std::cout << "voltage current temperature minutes: ";
    if (!(std::cin >> voltage >> current
              >> temperature >> minutes)) {
        std::cout << "Invalid input\n";
        return 1;
    }

    if (voltage <= 0.0 || current < 0.0 || minutes <= 0.0) {
        std::cout << "Invalid data\n";
        return 1;
    }

    double power{voltage * current};
    double energy{power * (minutes / 60.0)};

    std::cout << "Power(W): " << power << '\n';
    std::cout << "Energy(Wh): " << energy << '\n';

    if (voltage < 10.5 || temperature > 60.0
                  || current > 20.0) {
        std::cout << "State: Critical\n";
    } else if (voltage < 11.1 || temperature > 45.0
                        || current > 15.0) {
        std::cout << "State: Warning\n";
    } else {
        std::cout << "State: Normal\n";
    }
}
```

为什么必须先判断 Critical

Critical 条件与 Warning 条件可能同时为真。例如 $U = 10.0$ 时， 既有 $U < 10.5$，也有 $U < 11.1$。 若先判断 Warning，严重状态会被提前归入 Warning。 因此分支应按

$$\text{Critical} \longrightarrow \text{Warning} \longrightarrow \text{Normal}$$

从严重到轻微排列。

逻辑或的含义

Critical 条件为

$$(U < 10.5) \vee (T > 60) \vee (I > 20).$$

这里用逻辑或，是因为任意一个危险指标出现，就应进入 Critical。 程序中的写法对应：

```
voltage < 10.5 || temperature > 60.0 || current > 20.0
```

如果误用 &&，就会要求低电压、高温和过流三种危险同时发生， 大量单一故障将被漏报。

测试数据与手算

U/V	I/A	T/°C	t/min	P/W	E/Wh	**状态**
12.0	8.0	30	90	96	144	Normal
10.2	5.0	25	30	51	25.5	Critical
11.0	12.0	50	60	132	132	Warning
12.0	21.0	40	10	252	42	Critical
-1.0	5.0	25	30	–	–	Invalid data

以第一组为例：

$$P = 12.0 \times 8.0 = 96\,\text{W},$$

$$E = 96 \times \frac{90}{60} = 96 \times 1.5 = 144\,\text{Wh}.$$

三个状态指标分别为

$$12.0 \geq 11.1, \qquad 30 \leq 45, \qquad 8 \leq 15,$$

既不触发 Critical，也不触发 Warning，所以状态为 Normal。

第二组中：

$$10.2 < 10.5,$$

仅这一项就足以触发 Critical。第三组中 $U = 11.0 < 11.1$ 且 $T = 50 > 45$，但都没有达到 Critical 阈值，所以为 Warning。 第四组因 $I = 21 > 20$ 直接进入 Critical。

边界上的“严格”与“包含”

题目写的是 $U < 10.5$、$T > 60$、$I > 20$，因此 $U = 10.5$、$T = 60$、$I = 20$ 本身不触发 Critical。编程时不能擅自把 < 改成 <=，也不能把 > 改成 >=； 一个等号会改变边界数据的分类。

6 权威资料与核对说明

检索与使用原则

资料检索优先使用 C++ 工作草案及编译器/平台厂商的官方文档。 工作草案网页不是付费 ISO 标准正文，但可公开定位到相应条款； Microsoft 与 IBM 文档用于交叉核对“标准保证”和“具体实现选择”的差别。 本讲义只转述必要规则，没有大段复制来源文字。

[S1] Fundamental types
C++ Working Draft, Fundamental types [basic.fundamental]。 访问原文

[S2] sizeof
C++ Working Draft, Sizeof [expr.sizeof]。 访问原文

[S3] 乘、除与余数
C++ Working Draft, Multiplicative operators [expr.mul]。 访问原文

[S4] 前置自增/自减
C++ Working Draft, Increment and decrement [expr.pre.incr]。 访问原文

[S5] 后置自增/自减
C++ Working Draft, Increment and decrement [expr.post.incr]。 访问原文

[S6] 赋值与复合赋值
C++ Working Draft, Assignment and compound assignment operators。 访问原文

[S7] 关系运算符与链式比较
C++ Working Draft, Relational operators [expr.rel]。 访问原文

[S8] 宏替换
C++ Working Draft, Macro replacement [cpp.replace]。 访问原文

[S9] 标识符与保留名字
C++ Working Draft, Identifiers [lex.name]。 访问原文

[S10] 标识符（MSVC）
Microsoft Learn, Identifiers (C++)。 访问原文

[S11] ILP32 与 LP64 数据模型
IBM Documentation, ILP32 and LP64 data models and data type sizes。 访问原文

[S12] 字符与字符串字面量
Microsoft Learn, String and character literals (C++)。 访问原文

[S13] 逻辑与及短路
C++ Working Draft, Logical AND operator [expr.log.and]。 访问原文

[S14] 逻辑或及短路
C++ Working Draft, Logical OR operator [expr.log.or]。 访问原文

[S15] if 语句
C++ Working Draft, The if statement [stmt.if]。 访问原文

[S16] 内置类型（MSVC）
Microsoft Learn, Built-in types (C++)。 访问原文

—— 讲义结束｜建议先独立完成题目，再按“答案速查 → 详细解析”的顺序复盘 ——